Лабораторная работа 11. Наблюдаемость ML-сервиса

Версия: 2026–2027 учебный год Формат: команда 2–5 человек с индивидуальной сдачей Продолжительность: 2 академических часа и самостоятельная работа Предшествующая работа: ЛР10 «MLflow Registry и управляемый выпуск модели»

1. Цель

Создать воспроизводимый контур наблюдаемости ML-сервиса: ограниченные HTTP-метрики, JSON-логи, Prometheus scrape и rules, provisioned Grafana dashboard, формализованные SLI/SLO, контролируемый alert-сценарий и runbook, связывающий запрос с model name/version и release evidence.

2. Результаты обучения

Каждый студент должен уметь:

- различать metrics, logs, traces и evidence;
- выбирать Counter, Gauge и Histogram по смыслу;
- ограничивать cardinality labels;
- вычислять rate, error ratio и p95 через PromQL;
- объяснять bucket, окно и counter reset;
- находить JSON-log по request ID;
- связывать запрос с resolved model version;
- формулировать SLI/SLO с population и окном;
- переводить alert через pending, firing и resolved;
- применять runbook и подтверждать восстановление.

3. Командный результат

Команда предоставляет tag `lr11-v1` и:

1. FastAPI service с `/healthz`, `/readyz`, `/predict`, `/metrics`;
2. HTTP counter и latency histogram с ограниченными labels;
3. prediction outcome metric и `model_info`;
4. JSON-log запроса с request ID, status, duration и model identity;
5. Compose stack app + Prometheus + Grafana;
6. version-controlled Prometheus config и rules;
7. provisioned datasource и dashboard минимум из шести панелей;
8. SLI/SLO document;
9. минимум один alert и проверяемый runbook;
10. общий baseline load report и индивидуальные load/incident reports;
11. tests и CI validation конфигурации;
12. issue, небольшие PR, review и индивидуальные evidence cards.

4. Проверенные версии

На 20 октября 2026 года starter project использует:

- `prometheus-client>=0.26,<1`;

- Prometheus `v3.15.0`;
- Grafana OSS `13.2.4`.

Перед семестром преподаватель проверяет stable releases и security notes. Плавающие image tags `latest` не применяются.

5. Запуск

```
cp .env.example .env
# задайте случайный локальный GRAFANA_ADMIN_PASSWORD и полный SOURCE_COMMIT
docker compose up --build --detach
docker compose ps
```

Локальные адреса:

- API: `http://127.0.0.1:8080`;
- Prometheus: `http://127.0.0.1:9090`;
- Grafana: `http://127.0.0.1:3000`.

Пароль не добавляется в Git. Порты привязаны к loopback; публикация учебного Grafana в интернет не требуется.

6. Проверка targets и provisioning

В Prometheus target `flight-delay-api` должен иметь состояние UP. В Grafana автоматически появляются datasource `Prometheus`, папка `MLOps` и dashboard `Flight Delay Service`.

Если dashboard создан только вручную, работа не принимается: source JSON и provisioning YAML должны находиться в tag `lr11-v1`.

7. Обязательные метрики

Метрика	Тип	Labels	Назначение
`http_requests_total`	Counter	`method`, `route`, `status`	request rate и errors
`http_request_duration_seconds`	Histogram	`method`, `route`	latency distribution и p95
`prediction_outcomes_total`	Counter	`outcome`, `model_version`	результат классификации и ошибки
`model_info`	Gauge	`model_name`, `model_version`, `source_commit`	identity активной модели
`up`	Gauge Prometheus	`job`, `instance`	доступность scrape target

Запрещённые labels: request ID, student ID, user ID, полный URL, timestamp, текст ошибки, входные признаки и probability.

8. Route template

Label `route` принимает шаблон `/predict`, `/readyz`, `/metrics` или значение `unmatched`. Не используйте сырой path с идентификаторами. Test должен доказать, что разные request ID не создают разные series.

9. JSON-лог

Для каждого HTTP-request сервис пишет одну JSON-строку:

```
{
  "event": "http_request",
  "request_id": "lr11-student-01-0001",
  "method": "POST",
  "route": "/predict",
  "status": 200,
  "duration_ms": 8.7,
  "model_name": "flight-delay-classifier",
  "model_version": "3"
}
```

Лог не содержит request body, признаки, токены, пароль Grafana и персональные данные. При ошибке сохраняется тот же request ID, который возвращён в header.

10. Связь с выпуском ЛР10

В `.env` задайте `MODEL_NAME`, `MODEL_VERSION` и `SOURCE_COMMIT` из принятого release manifest. Подтвердите одинаковые значения:

1. в manifest ЛР10;
2. в `model_info`;
3. в JSON-log;
4. в response `/predict` для model version.

Если локальный starter bundle используется без Registry, укажите `MODEL_VERSION=local` и объясните ограничение; для полного балла применяется exact version ЛР10.

11. Воспроизводимая нагрузка

Командный baseline:

```
python scripts/observability_load.py \
  --base-url http://127.0.0.1:8080 \
  --requests 200 \
  --invalid-every 10 \
  --student-id team-baseline \
  --output reports/observability/lr11-team-baseline.json
```

Сценарий создаёт ожидаемые 200 и 422, но не 5xx. Report содержит число запросов, статусы, mean, p50, p95 и max. Он не заменяет Prometheus: локальные client timings и server histogram сравниваются как разные точки измерения.

12. Индивидуальная нагрузка

Каждый участник выбирает собственные `--requests`, `--invalid-every` и `--student-id`, фиксирует ожидания до запуска и сохраняет отдельный report. Запуск всей команды под одним ID не принимается.

Нагрузка выполняется последовательно и ограниченно. Цель — получить наблюдаемый сигнал, а не провести стресс-тест общей машины.

13. PromQL: request rate

```
sum by (route) (rate(http_requests_total[5m]))
```

Evidence содержит query, окно, время выполнения, result и объяснение. Студент должен показать, почему `rate` применяется до `sum`.

14. PromQL: error ratio

```
sum(rate(http_requests_total{status=~"5.."}[5m]))
/
clamp_min(sum(rate(http_requests_total[5m])), 0.001)
```

422 анализируются отдельным query:

```
sum(rate(http_requests_total{status="422"}[5m]))
```

Не называйте 422 server failure без обоснования.

15. PromQL: p95

```
histogram_quantile(
  0.95,
  sum by (route, le) (rate(http_request_duration_seconds_bucket[5m]))
)
```

Label `le` обязателен для classic histogram. Студент объясняет, почему p95 зависит от buckets и не равен максимуму.

16. Dashboard

Обязательные панели:

1. request rate;
2. 5xx ratio;
3. p95 latency by route;
4. prediction outcomes by model version;
5. target `up`;
6. resolved `model_info`.

У каждой панели есть title, query, единица, окно и понятная legend. Dashboard JSON проходит parse и хранится в Git. UI-изменения экспортируются обратно в source либо не учитываются.

17. SLI/SLO

Создайте `docs/observability/sli-slo.md` минимум с двумя SLI:

```
Availability SLI = valid /predict requests without 5xx / all valid /predict requests.
Latency SLI = fraction of valid /predict requests completed within 500 ms.
```

Для каждого SLO укажите population, good/bad event, target, окно, минимум трафика, PromQL и ограничение измерения. Учебный пример: availability >=99% и p95 <=500 ms за 15 минут при минимум 50 запросах.

18. Alert rules

Starter содержит:

- `FlightDelayServiceDown`: `up==0`, `for: 1m`;
- `FlightDelayHigh5xxRatio`: ratio >5%, `for: 2m`;
- `FlightDelayHighP95Latency`: p95 >500 ms, `for: 2m`.

Команда сохраняет один общий baseline; каждый студент отвечает за один alert или отдельную модификацию порога/окна. Alert должен иметь severity, team, summary и runbook annotation.

19. Контролируемые сценарии

Допустимые индивидуальные сценарии:

Сценарий	Ожидаемый сигнал	Восстановление
остановить только `app`	`up=0`, Down pending/firing	запустить app, дождаться resolved
временный тестовый 500 endpoint в отдельной ветке	5xx ratio	отменить commit/ветку
контролируемая задержка в отдельной ветке	p95 alert	убрать задержку
неверный model path	`/readyz=503`, not-ready outcomes	вернуть путь и рестарт
Prometheus не видит target	scrape failure	исправить config/network

Нельзя удалять чужие volumes, портить общий dashboard, бесконечно генерировать нагрузку или публиковать сервис наружу.

20. Alert lifecycle

Для своего сценария участник фиксирует:

1. baseline и release identity;
2. момент введения отказа;
3. pending;
4. firing;

5. выполненные шаги runbook;
6. восстановление;
7. resolved;
8. краткую причину и профилактику.

Если семинар ограничен по времени, преподаватель может уменьшить `for` в отдельной учебной ветке; изменение фиксируется и не попадает молча в tag.

21. Runbook

`docs/runbooks/flight-delay-api.md` должен содержать область, первые проверки, диагностику каждого alert, безопасные действия и критерий завершения. Команда дополняет runbook конкретными командами и владельцами.

Фраза «перезапустить всё» не считается runbook. Перезапуск допустим только после фиксации evidence и объяснения гипотезы.

22. Тестирование и CI

Обязательные проверки:

1. `/metrics` содержит обязательные метрики;
2. успешный `/predict` создаёт series с route template и status 200;
3. invalid request создаёт status 422 без request ID label;
4. JSON-log parseable и содержит model identity;
5. JSON-log не содержит payload;
6. dashboard JSON parseable;
7. Compose configuration валидна;
8. `promtool check config` проверяет config и rules;
9. image tags Prometheus/Grafana не плавающие;
10. `.env` не отслеживается Git.

CI не обязан поднимать долгоживущий Grafana. Он проверяет tests, Compose, Prometheus config/rules и сборку service image.

23. Командные роли

Для команды 2–5 человек:

- владелец application metrics и cardinality;
- владелец JSON logs/correlation/privacy;
- владелец PromQL и SLI/SLO;
- владелец Grafana provisioning/dashboard;
- владелец alerts/runbook/incident drill.

В малой команде роли совмещаются, но артефакты различаются. Роли ротируются относительно ЛР10.

24. Индивидуальная сдача участника команды

Командный результат: tag `lr11-v1`, observability stack, dashboard, rules, runbook и baseline report.

Каждый участник заполняет собственную evidence card. Совпадающие карточки не принимаются.

Поле	Что указать
Студент	ФИО, группа, команда, GitHub-логин
Лабораторная	ЛР11 и tag `lr11-v1`

Поле	Что указать
Роль	роль в ЛР11 и отличие от ЛР10
Задача	issue с наблюдаемым критерием
Личный артефакт	metric, query, panel, log schema, alert, runbook или test
Нагрузка	команда, параметры и личный JSON-report
Evidence	PromQL, окно, результат, request ID и model version
Отказ	личный сценарий, pending/firing/resolved и восстановление
Решение	labels/buckets/window/threshold и рассмотренная альтернатива
Риск	cardinality, PII/secrets, alert fatigue или неверный SLI
Использование ИИ	инструмент и самостоятельно проверенные утверждения

Минимальный допуск

1. Командная версия зафиксирована tag `lr11-v1`.
2. Участнику назначена собственная issue.
3. Есть содержательный личный артефакт и отдельный report.
4. Изменение прошло PR и review другого участника.
5. Evidence содержит query, окно, request ID и model identity.
6. Автоматические проверки успешны либо исключение письменно принято преподавателем.
7. Секреты, request bodies и персональные данные отсутствуют.

25. Индивидуальная защита

За 3–4 минуты участник:

1. показывает личный артефакт в tag;
2. запускает свой профиль нагрузки;
3. выполняет назначенный PromQL query;
4. находит JSON-log по request ID;
5. связывает log и `model_info` с release manifest;
6. объясняет labels, окно и тип метрики;
7. воспроизводит отказ или разбирает сохранённый lifecycle;
8. выполняет шаги runbook и подтверждает восстановление.

Преподаватель может изменить окно, threshold, долю invalid requests или попросить классифицировать новый label как безопасный/опасный.

26. Повторная сдача и стоп-факторы

Повторная сдача выполняется через issue, отдельный PR, новый load/incident report и обновлённую evidence card. История первой попытки сохраняется.

Стоп-факторы:

- request ID, user ID или raw path используется как label;
- dashboard создан только вручную;
- лог содержит признаки, пароль, токен или персональные данные;
- p95 вычисляется без histogram buckets или `le`;
- alert не имеет окна/владельца/runbook;
- общий скриншот заменяет query и report;

- model version не связана с выпуском;
- один incident report выдан всей команде;
- отказ разрушает общую среду или evidence;
- студент не может объяснить собственный label/query/threshold.

27. Критерии оценивания

Критерий	Баллы	Полное выполнение
Метрики и cardinality	2	корректные типы, names, labels и tests
Logs и release correlation	2	JSON, request ID, model identity, privacy
PromQL, dashboard и SLI/SLO	2	queries корректны, provisioning воспроизводим
Alert и runbook	2	lifecycle доказан, восстановление проверено
Индивидуальная защита	2	личная нагрузка, артефакт, диагностика и решение

Максимум: 10 баллов. Рекомендуемый порог зачёта: 6 баллов при отсутствии стоп-факторов.

28. Чек-лист

- □ Команда состоит из 2–5 человек, роли ротированы.
- □ Versions инструментов проверены и image tags фиксированы.
- □ Prometheus target имеет состояние UP.
- □ Dashboard и datasource provisioned из Git.
- □ Metrics используют route template и ограниченные labels.
- □ JSON-log parseable и не содержит payload/secrets.
- □ Release identity согласована с ЛР10.
- □ Baseline и личные load reports сохранены.
- □ Rate, 5xx ratio и p95 PromQL воспроизводимы.
- □ SLI/SLO содержат population, target и окно.
- □ Alert прошёл pending/firing/resolved или сохранён эквивалентный evidence.
- □ Runbook завершился проверкой восстановления.
- □ CI и tests успешны.
- □ У каждого участника отдельная evidence card.
- □ Итоговая версия отмечена tag `lr11-v1`.

29. Связь с ЛР12

В ЛР12 временные окна, model version и release manifest будут использованы для мониторинга drift, delayed labels, coverage и качества. Service health останется отдельным слоем: доступный API может обслуживать деградировавшую модель, а качественная модель может быть недоступна из-за отказа сервиса.

30. Официальные источники

- Prometheus instrumentation practices: https://prometheus.io/docs/practices/instrumentation/
- Prometheus query functions: https://prometheus.io/docs/prometheus/latest/querying/functions/
- Prometheus alerting rules: https://prometheus.io/docs/prometheus/latest/configuration/alerting_rules/
- Grafana provisioning: https://grafana.com/docs/grafana/latest/administration/provisioning/

- Grafana Docker installation: https://grafana.com/docs/grafana/latest/setup-grafana/installation/docker/
- Python Prometheus client releases: https://pypi.org/project/prometheus-client/